

Truy xuất tài liệu và Knowledge Graph theo thời gian cho AI Agent của GSM

1. Tổng hợp điều hành

Hướng phù hợp nhất cho dự án sáu tuần là giữ Graphiti làm lớp đồ thị thời gian, nâng chất lượng truy xuất tài liệu, rồi bổ sung lớp chọn bằng chứng kết hợp tài liệu-KG. Đóng góp R&D nên nằm ở khả năng tìm đủ bằng chứng đúng người, đúng chính sách và đúng thời điểm trong một ngân sách context và độ trễ xác định. Chưa có bằng chứng đủ mạnh để khuyến nghị thay Graphiti bằng một framework memory khác.

Landscape có bốn nhóm hữu ích: retrieval nhiều giai đoạn trên văn bản; graph-guided retrieval để nối các bằng chứng; memory construction để biến sự kiện thành cấu trúc có thể tìm kiếm; và temporal retrieval để chọn phiên bản sự thật thích hợp. Chúng giải quyết các lỗi khác nhau. Một embedding tốt hơn không sửa được định danh tài xế sai; một KG đúng không bảo đảm lấy được điều khoản ngoại lệ trong SOP; một câu trả lời trôi chảy không chứng minh rằng context đủ bằng chứng.

Trong ba seed, **Graphiti/Zep là baseline cần kiểm toán kỹ; A-Mem cung cấp ý tưởng liên kết và cập nhật memory có thể tách ra tái sử dụng; MemGPT là nền tảng quản lý context và công cụ memory.** KG²RAG gần nhất với vấn đề nối graph và đoạn văn. HippoRAG 2 hữu ích cho associative retrieval; IRCOT cho truy xuất nhiều bước có điều kiện; TG-RAG cho thiết kế đánh giá sau cập nhật theo thời gian. Các hệ thống này có mục tiêu và benchmark khác nhau, nên không tạo thành một bảng xếp hạng trực tiếp. [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#) [\[6\]](#) [\[11\]](#)

Thành phần	Ưu tiên đề xuất	Điều phải chứng minh
Document retrieval	BM25 + dense multilingual; RRF; reranker giới hạn số ứng viên; metadata và phiên bản tài liệu	Tăng recall của bộ bằng chứng và QA trên truy vấn tiếng Việt, không chỉ similarity
KG retrieval	Định danh bằng khóa nghiệp vụ; lọc thời gian; mở rộng 1-2 hop có giới hạn; bảo toàn nguồn	Không trộn tài xế, không ghép trạng thái thuộc các thời điểm không tương thích
Hybrid context	Ghép fact tài xế với điều khoản áp dụng; chọn evidence bundle theo token budget	Đủ cả dữ kiện và quy tắc, giữ được bằng chứng cầu nối và ngoại lệ
Memory construction	Giữ event gốc; tách fact, aggregate và giả thuyết; cập nhật có phiên bản	Sửa dữ liệu đến muộn mà vẫn truy vấn được lịch sử
Scale	Giới hạn fan-out; index khóa/thời gian; cập nhật cục bộ; cache có phiên bản; đo ingestion riêng	Đường cong chất lượng-chi phí khi tăng chunks, edges, events và concurrency

Không nên dành phần lớn thời gian để tái tạo MemGPT, huấn luyện graph foundation model, xây toàn bộ framework memory mới hoặc chạy community summarization trên mọi sự kiện. RAPTOR, GraphRAG toàn cục và GNN retrieval đáng đọc, nhưng chỉ triển khai nếu một nhóm truy vấn thực sự đòi hỏi chúng và baseline đơn giản đã bộc lộ lỗi tương ứng.

Độ chắc chắn của khuyến nghị: cao về thứ tự thử nghiệm và nhu cầu đánh giá độc lập; trung bình về lợi ích của graph-guided evidence packing; chưa xác định về mức tăng điểm, SLA hoặc cấu hình hạ tầng tại GSM. Báo cáo là tổng hợp nghiên cứu đến 14/09/2026; chưa chạy lại benchmark, chưa đọc mã triển khai nội bộ và không có dữ liệu production GSM.

Mục lục

1. Tổng hợp điều hành	1
2. Định nghĩa bài toán và chuẩn bằng chứng	3
3. Taxonomy phục vụ quyết định triển khai	4
4. Ma trận tài liệu nghiên cứu	5
5. Phân tích sâu 12 paper và hệ thống	6
6. Graphiti so với các lựa chọn liên quan	12
7. Kết quả tổng hợp về document retrieval — RQ1	13
8. Kết quả tổng hợp về KG và driver memory — RQ2, RQ3, RQ4	14
9. Kết hợp document và KG — RQ5	15
10. Scalability — RQ6	16
11. Kế hoạch đánh giá có thể thực hiện	19
12. Khoảng trống và kết quả nghiên cứu không đồng nhất	21
13. Ưu tiên công nghệ và rủi ro	22
14. Kế hoạch sáu tuần — 288 giờ	24
15. Thứ tự đọc đề xuất	24
16. Khuyến nghị cuối cùng	25
17. Sources	26

2. Định nghĩa bài toán và chuẩn bằng chứng

2.1. Đối tượng tối ưu

Đầu vào là truy vấn tác vụ, danh tính/phạm vi được phép truy cập, thời điểm nghiệp vụ và trạng thái quan sát của hệ thống. Đầu ra của retrieval là **một bộ bằng chứng có nguồn và quan hệ hỗ trợ**, trước khi LLM lập câu trả lời hoặc chọn hành động. Đo riêng construction, retrieval, context selection và reader/agent để biết lỗi nằm ở đâu.

Ví dụ tổng hợp, không mô tả quy định thật của GSM: “Tại ngày 12/8, tài xế D017 có đủ điều kiện nhận khoản hỗ trợ X không?” Câu trả lời có thể cần vùng hoạt động của D017 trong ngày đó, số sự kiện hợp lệ trong một cửa sổ, phiên bản chính sách X có hiệu lực và một điều khoản loại trừ. Tìm đúng tên chính sách nhưng thiếu ngoại lệ vẫn là retrieval thất bại.

Họ truy vấn	Bằng chứng bắt buộc	Dạng lỗi cần tách
Chính sách/SOP	Điều khoản, phạm vi, ngoại lệ, phiên bản	Đúng chủ đề nhưng sai điều kiện áp dụng
Trạng thái tài xế	Entity ID, fact có hiệu lực, nguồn cập nhật	Nhầm người; lấy trạng thái hiện tại cho câu hỏi lịch sử
Hồ sơ hành vi	Events, cửa sổ tính toán, mẫu số và quy tắc aggregate	Nhận xét thiếu căn cứ; aggregate không tái lập được
Chính sách áp dụng cho tài xế	Fact và quy tắc tương thích theo thời gian	Hai mảnh bằng chứng riêng đúng nhưng kết hợp sai
Tổng hợp/xu hướng	Phạm vi tập dữ liệu, chuỗi thời gian, bằng chứng đại diện	Suy rộng từ vài fact được retrieval
Không đủ thông tin	Bằng chứng thiếu, mâu thuẫn hoặc chưa quan sát	Biến “không thấy” thành “không có”

2.2. Phân biệt bốn loại thời gian

Event time là lúc sự kiện xảy ra. **Valid time** là khoảng fact đúng trong nghiệp vụ. **Transaction time** là khoảng hệ thống lưu/biết phiên bản fact đó. **Ingestion time** là thời điểm một bản ghi đi vào pipeline; nó có thể khác lúc hoàn tất xử lý hoặc lúc fact có thể tìm kiếm. Trong thiết kế đề xuất, dùng khoảng nửa mở [from, to) và timezone thống nhất.

Ví dụ tổng hợp: chuyển vùng có hiệu lực 01/08 nhưng hệ thống nhận ngày 03/08. Một bản sửa nhận ngày 05/08 xác nhận ngày hiệu lực đúng là 02/08. “Thực tế tại 01/08 là gì, theo dữ liệu hiện biết?” khác “Hệ thống đã biết gì vào 03/08?”. Bốn timestamp không tự động bảo đảm mọi query có ngữ nghĩa bitemporal đúng; cần ánh xạ rõ field, quy tắc đóng khoảng và xử lý bản sửa.

Đây là yêu cầu thiết kế của bài toán. Graphiti/Zep cung cấp điểm xuất phát với thời gian ở thế giới sự kiện và trong hệ thống, nhưng tính đúng đắn phải được kiểm tra trên chuỗi cập nhật, đặc biệt dữ liệu đến không theo thứ tự. ^[1]

2.3. Cách đọc kết quả nghiên cứu

Ưu tiên bài gốc, proceedings chính thức và repository tác giả. Bài được peer review vẫn có thể chỉ được nhóm tác giả tự đánh giá. Kết quả từ công ty phát triển sản phẩm được ghi là **author/vendor evaluation**, không xem là replication độc lập. “Có mã công khai” không đồng nghĩa “đã chạy được ở schema GSM”.

Năm preprint và năm venue được phân biệt trong ma trận. “NR” nghĩa là không tìm thấy hoặc chưa xác minh thông số cần thiết trong nguồn đã đọc, không có nghĩa tác giả chứng minh nó bằng không. ReasonIR được đối chiếu abstract, metadata và code; phụ lục thực nghiệm chưa được kiểm toán đầy đủ. IA-RAG chỉ được xem ở mức phát hiện một hướng mới năm 2026. Không dùng hai mục này để đưa ra khẳng định về production.

3. Taxonomy phục vụ quyết định triển khai

Nhóm	Cơ chế / câu hỏi chính	Nguồn tiêu biểu	Vai trò trong dự án
Lexical và dense	Khớp tên/mã và ngữ nghĩa; hợp nhất ứng viên	BGE-M3, SPLADE, BEIR, RRF	Baseline tài liệu mạnh, dễ chẩn đoán
Reranking và late interaction	Xét tương tác query-passage chi tiết	ColBERTv2, PLAID, BGE reranker	Tăng precision sau khi đủ recall
Reasoning-aware retrieval	Query thiếu từ khóa; cần bước trung gian	IRCoT, ReasonIR, BRIGHT	Thử sau khi biết loại câu hỏi baseline bỏ sót
Hierarchical document retrieval	Tìm cả chi tiết lẫn mức khái quát	RAPTOR, Contextual Retrieval	Bắt đầu với metadata và parent section
Graph fact retrieval	Chọn entity, relation đúng và gần query	Graphiti, LightRAG	Lớp truy xuất hồ sơ, sự kiện
Reasoning subgraph	Bảo toàn đường nối và tập evidence cần thiết	HippoRAG 2, G-Retriever, GFM-RAG	Mở rộng có giới hạn và cắt graph theo query
Temporal retrieval	Lọc trạng thái, lịch sử và cập nhật	Graphiti, TG-RAG, IA-RAG	Năng lực cốt lõi cần benchmark riêng
Memory construction	Event → fact → liên kết → tổng hợp	A-Mem, Hindsight, Mem0	Học ý tưởng, tái dùng có chọn lọc
Agent memory control	Khi nào đọc/ghi, context nào giữ	MemGPT	Khung tích hợp và giới hạn tool loop
Graph-text hybrid	Graph dẫn tới chunk; chunk chứng minh fact	KG ² RAG, GraphRAG local search	Trọng tâm R&D kết hợp hai nguồn

Các nhóm chống lẫn có chủ đích. Graph tạo từ tài liệu và KG tài xế có cấu trúc là hai trường hợp khác nhau: trong trường hợp thứ nhất, graph thường đóng vai trò index phụ; trong trường hợp thứ hai, graph còn chứa trạng thái nghiệp vụ độc lập. Không mặc định rằng một benchmark GraphRAG đã kiểm tra cả hai.

4. Ma trận tài liệu nghiên cứu

Tệp đi kèm **GSM-paper-matrix.xlsx** chứa 27 mục và 24 cột: ID, tên, năm, venue/trạng thái, nhóm, vấn đề, ý tưởng, representation, retrieval, temporal, incremental update, scale thực nghiệm, retrieval metrics, downstream metrics, latency/cost, code, độ khó, điểm phù hợp, giải thích điểm, hạn chế, tier, mức kiểm chứng, paper URL và ghi chú nguồn. Tab Sources cung cấp nguồn paper/code/docs trực tiếp.

Điểm phù hợp là nhận định cho bài toán này, không phải chất lượng học thuật: **5** đóng góp trực tiếp cho baseline hoặc đánh giá bắt buộc; **4** hữu ích nếu thích nghi; **3** tùy loại truy vấn/tài nguyên; **2** chủ yếu nền tảng; **1** ít liên hệ. Độ khó giả định một kỹ sư, sáu tuần và tái sử dụng code.

ID	Paper / hệ thống	Điểm	Lý do quyết định
P01	Zep / Graphiti	5	Baseline temporal KG hiện có
P02	A-Mem	4	Tách được dynamic linking/consolidation
P03	MemGPT	2	Quản lý context, ít giải quyết ranking trực tiếp
P04	KG ² RAG	5	Graph-guided chunk retrieval và organization
P05	HippoRAG 2	4	Associative retrieval; cần kiểm soát graph growth
P06	IRCoT	4	Hợp với multi-hop có bước tìm tiếp
P07	RAPTOR	3	Hữu ích cho tài liệu dài, chi phí cập nhật cần cân nhắc
P08	Microsoft GraphRAG	3	Local evidence mapping hữu ích; global summary tùy nhu cầu
P09	LightRAG	4	Hybrid entity/relation/chunk, có code thực dụng
P10	G-Retriever	4	Ý tưởng chọn subgraph theo ngân sách
P11	TG-RAG	5	Protocol thời gian và cập nhật sát khoảng trống hiện tại
P12	Hindsight	4	Tách evidence/inference và kết hợp nhiều retriever
P13	BGE-M3	5	Ứng viên multilingual baseline có thể dùng sẵn
P14	SPLADE	3	Learned sparse cho lexical mismatch; phải kiểm tra tiếng Việt
P15	ColBERTv2	3	Late interaction đổi chất lượng lấy index lớn hơn
P16	PLAID	3	Có bằng chứng scale IR lớn, nhưng tích hợp phụ thuộc ColBERT
P17	ReasonIR	3	Thử checkpoint khi reasoning retrieval là nút thắt

ID	Paper / hệ thống	Điểm	Lý do quyết định
P18	GFM-RAG	3	Pretrained graph retriever; chi phí thích nghi cao
P19	Mem0	3	Baseline memory extraction/update để đối chiếu
P20	Contextual Retrieval	4	Contextual prefix/metadata để thử có kiểm soát
P21	BEIR	4	Thiết kế đánh giá retriever trên nhiều domain
P22	BRIGHT	5	Chẩn đoán retrieval cần suy luận
P23	LongMemEval	5	Đánh giá temporal, update, multi-session, abstention
P24	LoCoMo	3	Liên hệ memory seeds; không đại diện vận hành GSM
P25	CRAG	4	Tác vụ có web và KG, độ mới và câu hỏi khó
P26	Reciprocal Rank Fusion	5	Thành phần fusion đơn giản cần baseline
P27	IA-RAG	3	Temporal interval reasoning mới, chỉ đọc bổ sung

5. Phân tích sâu 12 paper và hệ thống

5.1. Zep / Graphiti — nền tảng temporal, không phải kết luận về production

Vấn đề và cơ chế. Zep tổ chức episode gốc, entity/fact và community. Ingestion dùng LLM trích xuất, tìm entity tương tự, giải quyết trùng lặp và mâu thuẫn; fact giữ provenance và thời gian. Retrieval phối hợp semantic, lexical và graph search, rồi rerank để tạo context. Trong LongMemEval S, cùng GPT-4o, tác giả báo cáo accuracy 60,2% → 71,2%; thời gian phản hồi 28,9 → 2,58 giây. Đây là thời gian toàn phản hồi trong thiết lập đó, không phải retrieval p95. ^[1]

Khả năng hiện hành. Graphiti OSS và Zep managed là hai sản phẩm triển khai khác nhau. Tài liệu search hiện có RRF, MMR, node-distance, episode-mentions và cross-encoder; cross-encoder có thể dùng client LLM hoặc BGE. add_triplet cho phép đưa cấu trúc định sẵn nhưng vẫn thực hiện deduplication. Tài liệu community hiện mô tả Leiden và cập nhật tùy chọn, khác mô tả label propagation trong bài ban đầu. ^{[28] [29] [31] [32]}

Vì sao có thể hiệu quả — phân tích. Entity linking giúp gom phát biểu khác nhau về cùng người; timestamp giúp loại fact cũ; episode cho phép quay lại chứng cứ. Nhưng lợi ích phụ thuộc vào extraction và identity resolution đúng. Semantic deduplication sai giữa hai tài xế có tên giống nhau có thể làm hỏng cả lịch sử; reranking không sửa được graph đã gộp sai.

Đánh giá và giới hạn. LongMemEval S có 500 câu hỏi, history trung bình khoảng 115 nghìn token. Bài Zep do tác giả hệ thống thực hiện; không phải thử tải đồ thị tài xế hàng triệu entity. Một số so sánh dùng số liệu công bố trước, và MemGPT chưa được tái lập thành công trên LongMemEval trong nghiên cứu này. Không suy ra hệ số tăng tốc của dịch vụ nội bộ từ các số trên. ^[1]

Scale và triển khai — suy luận/đề xuất. Chi phí ingestion có thể do số lần extraction, resolution và contradiction checks chi phối, chứ không chỉ graph database. Với dữ liệu đã có schema, ưu tiên ánh xạ xác định, idempotency key và kiểm tra khoảng thời gian trước khi dùng LLM. Đo số lời gọi thực tế của phiên bản được pin, kể cả API nhập triple. Giới hạn neighborhood, không tự động bật community update trên mọi event, và lưu event gốc để sửa lịch sử.

Áp dụng trực tiếp. Giữ Graphiti, pin commit/package, LLM, embedding, reranker và backend. Viết một lớp query contract có driver_id, valid_at, known_as_of, source_ids và token budget; chứng minh từng trường được thực thi thay vì chỉ đưa vào prompt.

5.2. A-Mem — tái dùng liên kết động có kiểm soát

Vấn đề và cơ chế. A-Mem xây note gồm nội dung, context, keywords/tags và embedding. Note mới tìm hàng xóm tương tự; LLM quyết định link và cập nhật mô tả memory liên quan. Khi truy vấn, lấy note bằng semantic retrieval. Trên LoCoMo với GPT-4o-mini, multi-hop F1 là 45,85 so với MemGPT 25,52; loại cả linking/evolution còn 24,55, loại evolution còn 31,24. Đây là kết quả trong cùng thiết lập của tác giả, không phải điểm retrieval độc lập. ^{[2] [35]}

Vì sao có thể hiệu quả — phân tích. Context của một note được làm giàu bởi các sự kiện liên quan, nên một query không khớp từ ngữ của event gốc vẫn có đường tới bằng chứng. Ablation hỗ trợ vai trò của tổ chức memory trong benchmark đó. Với GSM, ý tưởng có giá trị nhất là gắn event vào episode nghiệp vụ và entity đã xác định, thay vì đưa mọi chuyển xe vào một mạng note tự do.

Đánh giá và giới hạn. Nghiên cứu dùng LoCoMo và DialSim, nhiều backbone; chưa chứng minh tải enterprise theo số driver/edge/event. Có timestamp không đồng nghĩa có valid-time và transaction-time đầy đủ. Chất lượng vẫn phụ thuộc strongly vào reader và loại câu hỏi; không nên chuyển một điểm F1 hội thoại thành kỳ vọng QA vận hành. ^[2]

Scale và triển khai — suy luận/đề xuất. Nếu mỗi note xem xét k hàng xóm, chi phí ứng viên tăng theo số note nhân k, cộng chi phí embedding/search và LLM. Link trùng và description ngày càng dài tạo write amplification. Chỉ consolidate sự kiện quan trọng hoặc theo cửa sổ, giữ nguồn và version của summary, đặt ngưỡng số link. Không để một summary được sinh lại xóa fact lịch sử.

Hữu ích khi thích nghi. Thử một ablation nhỏ: event/chunk gốc so với thêm mô tả ngắn dựa trên hàng xóm cùng driver. Lợi ích phải thể hiện ở evidence recall hoặc QA; đo cả số lần ghi lại và freshness lag. Không triển khai toàn bộ A-Mem song song với Graphiti trong sáu tuần.

5.3. MemGPT — memory hierarchy và quyền điều khiển context

Vấn đề và cơ chế. MemGPT giải quyết giới hạn context bằng main context và bộ nhớ ngoài, với công cụ đọc/ghi/tìm kiếm cùng cơ chế chuyển điều khiển. Recall memory lưu tương tác; archival memory chứa dữ liệu lâu dài. Bài kiểm tra hội thoại dài, document QA và key-value retrieval. Dự án phần mềm phát triển tiếp dưới tên Letta; code hiện tại không phải bản sao nguyên trạng thí nghiệm năm 2023. ^{[3] [36] [37]}

Vì sao có thể hiệu quả — phân tích. Agent có thể nạp phần context cần thiết và lưu phần chưa dùng ra ngoài, nên không phải nhét toàn bộ lịch sử vào mỗi prompt. Đây là một chính sách quản lý tài nguyên và tool use. Chất lượng bằng chứng vẫn do index, truy vấn và nội dung được nạp quyết định.

Đánh giá và giới hạn. DMR trong họ benchmark này sử dụng 500 cuộc hội thoại với nhiều phiên; không đại diện một KG nhiều triệu cạnh có fact bị sửa theo thời gian. Bài không cung cấp bằng chứng rằng cơ chế paging tự cải thiện entity resolution, temporal filtering hoặc ranking trên tài liệu tiếng Việt. ^[3]

Scale và triển khai — suy luận/đề xuất. Bộ nhớ ngoài lớn vẫn cần retrieval hiệu quả. Nhiều bước gọi memory tuần tự sẽ cộng độ trễ và có thể lập vô ích. Nên mượn cách phân tách state và công cụ:

agent gọi một interface retrieval nhất quán, nhận evidence có ID, và có giới hạn lượt gọi. Không để LLM tự quyết định giữ/xóa lịch sử nghiệp vụ có tính chuẩn xác.

Hữu ích về nền tảng. Đọc để thiết kế integration và quản lý context budget. Không chọn việc tái xây MemGPT làm đóng góp R&D chính; dùng agent framework sẵn có và tập trung vào retrieval phía sau công cụ.

5.4. KG²RAG — nối graph với bằng chứng dạng đoạn văn

Vấn đề và cơ chế. KG²RAG ánh xạ triple về chunk nguồn. Semantic retrieval chọn seed chunks; graph mở rộng bằng chứng liên quan; bước organization lọc và sắp xếp context. Trên HotpotQA fullwiki, answer F1 đạt 0,631 so với semantic retrieval có reranking 0,587. Trong distractor setting, bỏ organization tăng recall nhưng context dài hơn: khoảng 16,76 thay vì 8,11 chunks, trong khi answer F1 0,660 so với 0,663 của hệ đầy đủ. ^{[4] [38]}

Vì sao có thể hiệu quả — phân tích. Một đoạn câu nối có thể không gắn query về embedding nhưng nối seed tới bằng chứng trả lời. Ngược lại, lấy mọi hàng xóm làm tăng recall mà đưa thêm nhiễu. Cơ chế cân cả expansion và selection; chỉ thêm graph vào vector search chưa đủ.

Đánh giá và giới hạn. Có HotpotQA distractor/fullwiki và biến thể đổi tên nhằm giảm lợi thế ghi nhớ sẵn. “Fullwiki” không tự chứng minh mọi hệ thống đã index cùng số passage; ma trận không gán số corpus chưa xác minh. Benchmark dùng graph tạo từ text, chưa chứng minh ghép một temporal operational KG độc lập với policy corpus. ^[4]

Scale và triển khai — suy luận/đề xuất. Expansion không giới hạn có thể tăng theo bậc branching factor lũy thừa số hop. Dùng relation whitelist, thời gian, ngưỡng degree và số chunk tối đa. Với GSM, policy thường không chứa tên tài xế: cần liên kết qua policy_id, service, region, eligibility dimensions và effective interval. Chỉ dùng trùng entity text sẽ bỏ lỡ quan hệ quan trọng này.

Áp dụng trực tiếp với thích nghi nhỏ. Đây là paper hybrid nên ưu tiên đọc và tái dùng ý tưởng. Giữ Graphiti, lưu mapping fact-episode-document/chunk, rồi so sánh seed-only, seed+expansion và seed+expansion+packing ở cùng token budget. Khi graph và text mâu thuẫn, trả cả nguồn với trạng thái mâu thuẫn để xử lý; không cho graph summary thay thế điều khoản gốc.

5.5. HippoRAG 2 — associative retrieval và chi phí của graph mở rộng

Vấn đề và cơ chế. HippoRAG 2 kết hợp graph phrase/triple với passage nodes. Query tìm triple và passage, recognition filter chọn seed, rồi Personalized PageRank lan truyền để xếp hạng passage. Thử nghiệm gồm bảy bộ QA. Trên MuSiQue với cấu hình Llama được báo cáo, Recall@5 tăng từ 69,7 lên 74,7 so với NV-Embed-v2. Bài còn đánh giá memory mở rộng theo các đợt corpus. ^{[5] [39]}

Vì sao có thể hiệu quả — phân tích. Passage node giữ liên hệ với nội dung gốc, còn phrase graph tạo đường liên tưởng xuyên đoạn. Recognition filter giảm seed sai trước propagation. Nhưng loại nhầm seed đúng là lỗi không dễ phục hồi; node phổ biến có thể chiếm ưu thế nếu trọng số và lọc không phù hợp.

Đánh giá scale. Với 22.849 passages của LV, một cấu hình Llama tạo khoảng 198 nghìn nodes và 3,36 triệu edges. Appendix báo indexing 99,5 phút, QA trung bình 1,2 giây và memory 9,9 GB, trong thiết lập có bốn H100; memory loại trọng số model. Các số này không phải p95 nhiều người dùng. ^[5]

Scale và triển khai — suy luận/đề xuất. Graph có thể lớn hơn nhiều lần corpus gốc, đặc biệt cạnh đồng nghĩa. Một iteration PPR chạm phần graph được duyệt; nếu chạy trên toàn graph, chi phí phụ thuộc số cạnh và số iteration. Local PPR trên graph đã cắt là một biến thể cần kiểm chứng, không được gọi là tái lập nguyên thuật toán. Cần đo mức tăng edge/chunk, peak RAM và thời gian cập nhật.

Hữu ích khi thích nghi. Tái dùng passage-fact mapping và retrieval có graph propagation cho một nhóm multi-hop. Không thêm synonym edge đại trà vào driver KG; ID nghiệp vụ và relation có kiểu có

giá trị hơn similarity giữa tên. Chỉ thử PPR sau bounded traversal baseline để xác định lợi ích thực tế của propagation.

5.6. IRCot — khi truy vấn thứ hai cần thông tin từ truy vấn thứ nhất

Vấn đề và cơ chế. IRCot xen kẽ retrieval với câu suy luận trung gian để tạo query tiếp theo. Bài dùng HotpotQA, 2WikiMultihopQA, MuSiQue và IIRC, so sánh retrieval đơn bước và các biến thể. Một kết quả đặc biệt liên quan R&D này: với GPT-3 trên IIRC, retrieval recall tăng khoảng 21 điểm nhưng QA không cải thiện. ^{[6] [40]}

Vì sao có thể hiệu quả — phân tích. Query ban đầu có thể thiếu entity cầu nối. Bằng chứng vòng đầu cung cấp tên hoặc điều kiện cần để truy vấn vòng hai. Nhưng retrieval thêm thông tin chỉ hữu ích nếu reader sử dụng được và các bước trung gian không mang giả định sai. Quan hệ giữa recall và task success vì vậy phải được đo, không giả định.

Đánh giá và giới hạn. Protocol của bài lấy 100 câu development và 500 test trên mỗi dataset, dùng nhiều bộ demonstration. Điều đó khác thử hàng nghìn truy vấn đồng thời trên corpus đang cập nhật. Không có đảm bảo về độ trễ và chi phí ở backend Graphiti hoặc tiếng Việt. ^[6]

Scale và triển khai — suy luận/đề xuất. H hop tuần tự thường cộng H lần retrieval và các bước LLM tương ứng; wall-clock không thể coi như chạy song song độc lập. Một bước sai có thể dẫn toàn bộ lượt sau lệch hướng. Đối với GSM, nên dùng một lượt follow-up có cấu trúc: bằng chứng nào thiếu, entity nào đã xác nhận, query bổ sung nào được phép. Trạng thái này không cần xuất diễn giải suy luận nội bộ cho người dùng.

Hữu ích khi thích nghi. Chỉ bật khi bộ bằng chứng thiếu một thành phần bắt buộc hoặc query đòi hỏi bridge. Đánh giá trên cùng query với single-pass; log số lượt, tỷ lệ được cứu và tỷ lệ bị làm hỏng. Dùng nếu lợi ích chỉ đến từ tăng token/context mà không vượt baseline cùng ngân sách.

5.7. RAPTOR — truy xuất nhiều mức trừu tượng

Vấn đề và cơ chế. RAPTOR chia đoạn, embedding, giảm chiều, soft clustering rồi sinh summary đệ quy; query có thể tìm trên nhiều tầng. Trên cấu hình kiểm soát UnifiedQA-3B/MBERT, QuALITY accuracy 54,9 → 56,6 và QASPER F1 36,23 → 36,70. Phụ lục scale khảo sát khoảng 12.500–78.000 input tokens. Đây là bằng chứng nhỏ hơn rất nhiều so với hàng triệu chunks. ^{[7] [41]}

Vì sao có thể hiệu quả — phân tích. Summary đặt các chi tiết rời rạc vào một khái niệm chung, giúp query ở mức mục tiêu tìm được tài liệu dù không gọi đúng thuật ngữ của đoạn. Nhưng summary có thể bỏ điều kiện hiếm, đặc biệt ngoại lệ hoặc mốc hiệu lực — chính những yếu tố quan trọng với SOP.

Đánh giá và giới hạn. Các dataset gồm NarrativeQA, QASPER và QuALITY; độ mạnh headline giữa các hệ thống không tương đương tác động của riêng indexing hierarchy. Bài có kiểm tra hallucination summary nhưng không thiết lập guarantee không lỗi. Không có temporal KG hoặc transaction-time semantics trong cơ chế cốt lõi. ^[7]

Scale và triển khai — suy luận/đề xuất. Xây summary thêm token LLM và storage ngoài chunks gốc; thay một đoạn có thể cần sửa ancestors hoặc phân cụm lại. Với SOP có cấu trúc rõ, parent section và tiêu đề được trích xuất xác định là baseline rẻ hơn cần thử trước. Summary chỉ nên dùng làm đường dẫn retrieval, câu trả lời vẫn trích đoạn gốc.

Tùy loại truy vấn. Chỉ chạy RAPTOR trên một tập tài liệu dài, ít đổi, có câu hỏi tổng hợp. So sánh với parent-child retrieval và metadata contextualization. Không đưa vào đường cập nhật mỗi chuyển xe hoặc mỗi thay đổi hồ sơ.

5.8. Microsoft GraphRAG — phân biệt global research và local implementation

Vấn đề và cơ chế. Bài GraphRAG ban đầu tập trung query tổng hợp toàn corpus: entity graph, community summaries, tạo đáp án từng community rồi hợp nhất. Hai corpus khoảng 1 triệu và 1,7 triệu token; đánh giá chủ yếu bằng LLM so sánh comprehensiveness/diversity. Implementation hiện có local search kết hợp entity, relationships và text units nguồn; không nên mô tả framework hiện tại là chỉ có global search. [\[8\]](#) [\[33\]](#) [\[42\]](#)

Vì sao có thể hiệu quả — phân tích. Global summary cung cấp độ phủ chủ đề mà top-k đoạn gần query dễ bỏ lỡ. Local search có ích khi entity là điểm tựa để tìm source text. Hai loại lợi ích khác nhau: “bao quát” không chứng minh “đủ bằng chứng để quyết định một điều kiện áp dụng”.

Đánh giá và giới hạn. Bài gốc không phải benchmark transaction-time driver KG. Thí nghiệm lịch sử báo indexing corpus podcast khoảng 281 phút với API GPT-4 Turbo trong cấu hình tác giả; không thể chuyển trực tiếp thành chi phí hiện nay. Venue của bản technical report được dẫn ở đây chưa được xác nhận từ proceedings, nên ghi arXiv. [\[8\]](#)

Scale và triển khai — suy luận/đề xuất. Entity extraction, community summary và map-reduce có thể làm LLM token cost lớn ở cả offline lẫn online. Khi tài liệu đổi, cần đo phần artifact nào phải tái tạo. Việc bản hiện hành có thêm chế độ truy vấn/cập nhật không chứng minh bitemporal semantics; ngược lại, cũng không nên gọi nó “không thể cập nhật”.

Hữu ích có chọn lọc. Mượn mapping graph-source và cách chọn local context. Đặt global summarization ở Tier 3 trừ khi stakeholder thực sự cần xu hướng toàn tổ chức. Không thay temporal driver KG bằng một graph được trích từ tài liệu chỉ để dùng cùng framework.

5.9. LightRAG — truy xuất ở mức entity và quan hệ/chủ đề

Vấn đề và cơ chế. LightRAG tạo entity/relation từ text, kết hợp truy xuất low-level và high-level rồi đưa source chunks vào context. Bài đánh giá bốn corpus UltraDomain khoảng 600 nghìn–5 triệu token, chủ yếu qua đánh giá cặp bởi LLM. So với GraphRAG, tỷ lệ ưu tiên tổng thể thay đổi theo corpus và có trường hợp dưới 50%; không phải thắng toàn diện. [\[9\]](#) [\[43\]](#)

Vì sao có thể hiệu quả — phân tích. Entity search phục vụ query cụ thể, relation/topic search hỗ trợ query khái quát. Hợp nhất hai góc nhìn có thể giảm bỏ sót so với một vector index. Tuy nhiên, một relation summary có độ tương đồng cao vẫn có thể thiếu source hoặc phiên bản đúng.

Đánh giá và giới hạn. Điểm judge preference không thay thế answer accuracy với gold evidence. Framework hiện tại đã bổ sung nhiều tùy chọn so với paper, vì vậy phải pin phiên bản khi tái lập. Incremental graph integration không đồng nghĩa lưu và truy vấn được tất cả phiên bản sự thật theo hai trục thời gian. [\[9\]](#) [\[43\]](#)

Scale và triển khai — suy luận/đề xuất. Chi phí phụ thuộc số entity/relation trích được, số lần hợp nhất mô tả và số chunk kéo theo. Update cục bộ chỉ tiết kiệm khi nó không lan rộng tới nhiều summaries. Nên đo graph growth và sửa/xóa tài liệu, thay vì chỉ thử thêm tài liệu mới.

Hữu ích khi thích nghi. Có thể dùng làm đối chứng graph-enhanced document retrieval nhỏ, hoặc tái dùng chiến lược tách truy vấn cụ thể và chủ đề. Không chuyển driver state sang LightRAG chỉ vì kết quả RAG trên corpus text; cần chứng minh temporal correctness tương đương baseline trước khi cân nhắc.

5.10. G-Retriever — truy xuất một subgraph có ích thay vì các fact rời

Vấn đề và cơ chế. G-Retriever gán prize từ similarity cho node/edge, chọn subgraph bằng Prize-Collecting Steiner Tree, dùng GNN tạo soft prompt kết hợp text cho LLM. GraphQA gồm ExplaGraphs, SceneGraphs và WebQSP. Trong một phép đo WebQSP, graph trung bình giảm từ khoảng 1.371 xuống 18 nodes, text từ 100.627 xuống 610 tokens. Đây là subgraph theo câu hỏi, không phải truy xuất trực tiếp toàn Freebase. ^{[10] [44]}

Vì sao có thể hiệu quả — phân tích. Việc tối ưu đồng thời relevance và kết nối giữ lại node cầu nối vốn có similarity thấp. Đây là khác biệt giữa danh sách top-k fact và một cấu trúc hỗ trợ reasoning. Ablation retrieval của bài cho thấy bước chọn graph có đóng góp trong benchmark; không suy ra PCST luôn tối ưu cho mọi tập evidence.

Scale và triển khai — suy luận/đề xuất. Phải tính similarity ứng viên và chạy solver; cần candidate pruning trước trên graph lớn. Graph dạng cây có thể bỏ chu trình hoặc hai thành phần rời nhưng cùng cần cho kết luận. Đặc biệt, policy evidence và driver facts có thể chưa có cạnh trực tiếp; không nên tạo cạnh không có căn cứ chỉ để ép connectedness.

Hữu ích khi thích nghi. Trong sáu tuần, mượn objective “relevance + bridge coverage – chi phí context”, thử heuristic bounded subgraph trước PCST/GNN. Temporal eligibility là ràng buộc cứng trước optimization. Full GNN training và soft-prompt integration chỉ hợp lý nếu đã có dataset graph QA đủ tin cậy và baseline retrieval còn hạn chế rõ.

5.11. TG-RAG — temporal retrieval và protocol cập nhật

Vấn đề và cơ chế. TG-RAG kết hợp graph relation có timestamp với cây thời gian và summary nhiều độ hạt; query chọn temporal/semantic subgraph hoặc summary. ECT-QA gồm 480 earnings transcripts của 24 công ty, giai đoạn 2020-2024, khoảng 1,58 triệu token; 1.005 câu cụ thể và 100 câu trừu tượng. Protocol dùng 384 tài liệu ban đầu, rồi thêm 96 tài liệu năm 2024. ^{[11] [45]}

Vì sao có thể hiệu quả — phân tích. Hai fact giống nhau ở hai kỳ được phân biệt về thời gian; summary từng giai đoạn hỗ trợ trend query. Protocol kiểm tra cả câu cũ sau update và câu mới, nên phát hiện memory “học mới nhưng phá cũ”. Đây là phần đáng tái dùng nhất cho GSM.

Đánh giá và giới hạn. Trong bảng tác giả, accuracy trên câu mới sau update là 0,617 so với HippoRAG 2 là 0,372. Chi phí update được báo cáo cao hơn HippoRAG 2 về token; không thể gọi là temporal improvement miễn phí. ECT-QA là benchmark do nhóm nghiên cứu xây, không phải đối chiếu trực tiếp Graphiti hay tải vận hành thực. ^[11]

Scale và triển khai — suy luận/đề xuất. Cập nhật time leaf và ancestors có thể giới hạn phần summary cần viết lại, nhưng late correction vẫn ảnh hưởng nhiều mức. Quan hệ theo thời gian không tự bảo đảm transaction-time audit. Cần thêm kiểm tra known-as-of, sự kiện đến muộn và khoảng hiệu lực chồng lấn.

Áp dụng trực tiếp cho đánh giá, thích nghi cho retrieval. Xây benchmark “before/after/update/correction” trước khi cân nhắc time hierarchy. Tận dụng temporal fields của Graphiti; chỉ thêm summary theo tuần/tháng khi cần truy vấn hành vi hoặc xu hướng, với nguồn và version rõ ràng.

5.12. Hindsight — tách bằng chứng khỏi suy diễn

Vấn đề và cơ chế. Hindsight tổ chức world facts, experiences, observations tổng hợp và opinions; cung cấp retain/recall/reflect. Recall kết hợp semantic, lexical, graph và temporal retrieval, sau đó fusion/reranking. Technical report báo LongMemEval S cùng backbone OSS-20B: full context 39,0% và hệ memory 83,6%. Có bài System Demonstrations ACL 2026 và code công khai. ^{[12] [34] [46]}

Vì sao có thể hiệu quả — phân tích. Nhiều đường retrieval giúp truy cập cả fact cụ thể, liên hệ entity và khoảng thời gian. Tách nguồn trực tiếp khỏi observation/ý kiến giúp giảm việc biến một diễn giải thành sự kiện chắc chắn. Đây là nguyên tắc hữu ích cho hồ sơ hành vi tài xế.

Đánh giá và giới hạn. Điểm 91,4% trong cấu hình mạnh hơn dùng backbone khác; một số baseline khác được nhập từ báo cáo trước, không phải tất cả đều được chạy cùng model. Không nên lấy 91,4 trừ 71,2 của Zep để tuyên bố hệ nào hơn trong GSM. Technical report không cung cấp một bộ ablation sạch để phân bổ toàn bộ lợi ích cho từng thành phần retrieval. ^[12]

Scale và triển khai — suy luận/đề xuất. Nhiều mạng memory, reranker và reflection làm tăng indexing, update và serving cost. Triển khai demo không chứng minh QPS enterprise. “Opinion” của agent không đồng nghĩa “đánh giá tài xế”; các đặc tính suy ra phải có cửa sổ dữ liệu, mức tin cậy, nguồn và thời điểm tính lại. Quan hệ được gắn nhãn causal cũng không phải bằng chứng xác lập nhân quả.

Hữu ích khi thích nghi. Mượn separation of evidence/inference và phối hợp các retriever. Không cần thay Graphiti bằng một memory bank khác. Đưa observation vào context như summary có thể truy vết, và luôn cho phép drill-down tới events gốc.

6. Graphiti so với các lựa chọn liên quan

Các cột dưới đây mô tả cơ chế đã công bố và nhận định tích hợp. **Chưa có phép thử chung đủ điều kiện để xếp hạng accuracy, latency hoặc chi phí của các lựa chọn trên schema GSM.**

Khía cạnh	Graphiti / Zep	TG-RAG	HippoRAG 2
Temporal	Fact có mốc thời gian thế giới/hệ thống; phải kiểm thử query contract	Temporal KG + time hierarchy; bitemporal audit chưa chứng minh	Không có cơ chế bitemporal cốt lõi
Construction	Episodes → entity/fact, resolution/invalidation	Text theo thời gian → relations + summaries	OpenIE phrases/triples + passage/synonym graph
Entity resolution	Candidate matching + xử lý duplicate	Phụ thuộc extraction/graph merging trong pipeline	Phrase/synonym association; không thay business ID
Retrieval	Lexical + vector + graph; nhiều reranker	Temporal/semantic scope, graph propagation và summary	Triple/passage seeding, recognition, PPR
Update	Thiết kế incremental; cân đo chi phí từng episode	New time nodes/ancestors; có protocol update	Có thử corpus tăng; không đồng nghĩa xử lý fact invalidation
Scale/latency	LongMemEval chủ yếu memory QA; không có tải GSM	480 documents; token/update được báo cáo	22.849 passages; graph vài triệu cạnh; QA trung bình theo hardware riêng
Tích hợp	Thấp-trung bình vì là baseline; nhiều backend/client cấu hình	Trung bình–cao; prototype temporal document RAG	Trung bình–cao; graph/index khác baseline

Nguồn cho các cơ chế và thử nghiệm: Zep/Graphiti, TG-RAG và HippoRAG 2. ^{[1] [28] [29] [11] [5]}

Khía cạnh	Graphiti làm baseline	KG ² RAG / LightRAG / GraphRAG local	Hindsight / A-Mem / Mem0
Vai trò	Dynamic operational KG	Graph-enhanced document retrieval	Memory extraction, linking và recall
Temporal	Có nền biểu diễn và invalidation	Không mặc định tương đương bitemporal driver KG	Có thời gian ở mức khác nhau; phải kiểm tra semantics
Hybrid	Cần chứng minh mapping tới policy evidence	Source chunk mapping là ý tưởng mạnh	Kết hợp nhiều đường memory; không tự tạo policy applicability join
Cập nhật	Native ingestion; LLM cost cần đo	Có hỗ trợ/tuyên bố update tùy framework/version	Add/update/consolidate có write amplification
Accuracy	Không suy từ số managed service sang OSS	Hotpot evidence QA hoặc LLM preference tùy bài	Chủ yếu conversational memory QA
Mức trưởng thành	Code + docs công khai; nội bộ đã dùng	Framework và research repo không cùng mức vận hành	Code công khai; production suitability chưa được kiểm chứng tại GSM
Quyết định	Giữ làm graph layer	Bổ sung cơ chế; chưa cần thay toàn stack	Tái dùng ý tưởng có ablation

Nguồn: KG²RAG, LightRAG, GraphRAG local search, A-Mem, Hindsight và Mem0. [\[4\]](#) [\[9\]](#) [\[33\]](#) [\[2\]](#) [\[12\]](#) [\[19\]](#)

Kết luận so sánh: temporal semantics và khả năng nối source evidence là hai trục khác nhau. Giữ lợi thế temporal của Graphiti, rồi mượn retrieval/packing từ hệ khác là phương án có phạm vi nhỏ và kiểm chứng được. Điểm yếu cần giải quyết trước là độ đúng dữ liệu và query contract, không phải thiếu một tên framework mới.

7. Kết quả tổng hợp về document retrieval — RQ1

7.1. Baseline nên đủ mạnh trước khi thêm graph

Đề xuất bắt đầu bằng chunk theo section, giữ heading, document ID, version, effective interval và applicability metadata. Chạy BM25 để giữ mã/quy định/tên riêng và dense multilingual để xử lý paraphrase, fusion ứng viên rồi rerank. BGE-M3 là ứng viên thực dụng: hỗ trợ dense, sparse và multi-vector trong một họ model multilingual; không có bằng chứng từ bài rằng nó tối ưu cho tiếng Việt nghiệp vụ GSM. BEIR cho thấy cần so sánh lexical baseline nghiêm túc và đo trade-off của reranking/late interaction. [\[13\]](#) [\[21\]](#)

Cấu hình khởi đầu để thử, không phải optimum: mỗi retriever lấy 30–50 ứng viên; RRF tạo tập hợp tối đa 60–100; rerank tối đa 40; context cuối giới hạn theo token và evidence coverage. Tune trên dev. Nếu reranker bỏ bằng chứng cầu nối, giữ một quota bridge/evidence type thay vì chỉ lấy top score. Dense-only, BM25-only và hybrid là đối chứng cần có trong cùng harness.

RRF dùng tổng $1 / (c + \text{rank})$ trên các danh sách thứ hạng. Nó tránh phải cộng trực tiếp điểm cosine với BM25, nhưng vẫn cần một định danh chung cho cùng đối tượng được rank. Thử $c = 60$ là điểm khởi đầu truyền thống, không phải bảo đảm tối ưu trên schema mới. [\[26\]](#)

7.2. Thêm context cho chunk trước khi dùng hierarchy đặt

Contextual Retrieval đưa một đoạn context ngắn trước chunk để embedding và lexical index hiểu vị trí của đoạn trong tài liệu. Báo cáo engineering của Anthropic nêu mức giảm tương đối 67% retrieval failures khi kết hợp contextualization và reranking trong thiết lập của họ; đó không phải tăng 67 điểm accuracy và không phải thử nghiệm peer reviewed độc lập. ^[20]

Thử theo thứ tự: title/section metadata xác định; parent section retrieval; sau đó mới LLM contextual prefix. Không đưa thông tin từ phiên bản chính sách tương lai vào prefix của phiên bản cũ. Lưu prefix như index aid, không thay text nguồn. Mỗi lần sửa tài liệu phải biết những chunk/prefix nào cần tái sinh.

7.3. Reasoning-aware không chỉ là query dài hơn

BRIGHT đánh giá retrieval cần lập luận trên 1.384 query thuộc 12 domain. ReasonIR huấn luyện retriever với query khó và hard negatives; abstract báo nDCG@10 29,9 không reranker, 36,9 có reranker. Đây là bằng chứng về retrieval trên benchmark đó, không chứng minh chuyển thẳng sang tiếng Việt hoặc temporal driver data. ^{[22] [17]}

Đề xuất: phân loại lỗi lexical mismatch, thiếu bridge entity, thiếu điều kiện áp dụng và thiếu phép tổng hợp. Chỉ rewrite query khi biết loại lỗi. Query expansion phải giữ nguyên phủ định, ngày và ID; một rewrite thêm giả định sai có thể tăng semantic recall nhưng làm sai toàn bộ tác vụ. Với câu cần một entity trung gian, thử một follow-up có cấu trúc theo IRCOT. Với câu cần cộng/đếm đầy đủ, dùng truy vấn dữ liệu xác định thay vì semantic top-k.

7.4. Learned sparse và late interaction ở Tier 2

SPLADE dùng biểu diễn sparse có expansion học được; ColBERTv2 dùng tương tác token và compression; PLAID tối ưu engine cho late interaction và bảo thử nghiệm tới 140 triệu passages. Đó là bằng chứng scale retrieval văn bản đáng chú ý, nhưng không bao gồm temporal updates hay agent workload của GSM. ^{[14] [15] [16]}

Chỉ thêm một ứng viên nếu BM25+dense+reranker còn bỏ sót ổn định và máy chạy đáp ứng được. SPLADE checkpoint tiếng Anh không mặc định thay thế BM25 tiếng Việt tốt hơn. ColBERT/PLAID cần tính lại index/storage, không chỉ so thời gian query. Không đồng thời thử cả SPLADE, ColBERT, ReasonIR và RAPTOR trong sáu tuần.

8. Kết quả tổng hợp về KG và driver memory — RQ2, RQ3, RQ4

8.1. Entity đúng trước, graph expansion sau

Đề xuất ưu tiên business key bất biến như driver_id; tên, số điện thoại hoặc biệt danh chỉ là thuộc tính/alias có lịch sử. Quy tắc resolution phải phân biệt “cùng tên” với “cùng người”. Không dùng LLM fuzzy merge khi khóa nghiệp vụ đã quyết định được. Nếu schema không có khóa ổn định, đó là câu hỏi phải giải quyết với bên cung cấp schema trong tuần đầu.

Fact retrieval trả các quan hệ phù hợp. Reasoning subgraph còn phải chứa cạnh nối, mốc thời gian và nguồn đủ để kiểm tra kết luận. Với câu hỏi “vì sao chính sách X áp dụng?”, top-k fact tài xế không đủ nếu thiếu relation nối tới tiêu chí X. Ý tưởng graph-guided context trong KG²RAG và subgraph selection trong G-Retriever hỗ trợ cách tiếp cận này; việc áp vào schema GSM là đề xuất cần thử. ^{[4] [10]}

Retrieval contract đề xuất: giải quyết ID và scope; xác định thời gian; lấy seed fact; mở rộng 1–2 hop theo relation whitelist; lọc temporal eligibility; lấy source chunks/events; rerank bundle; cắt theo

ngân sách. Nếu query yêu cầu đường dài hơn, nâng hop có điều kiện và đo phần tăng chi phí. Đừng đặt “hai hop” thành giới hạn ngữ nghĩa bất biến nếu gold evidence thực sự cần ba hop.

8.2. Tính đúng theo thời gian là ràng buộc, không chỉ một feature ranking

Với truy vấn tại thời điểm t , fact chỉ hợp lệ khi $\text{valid_from} \leq t < \text{valid_to}$, theo quy ước null là đầu/cuối mở đã định nghĩa. Với known-as-of a , còn cần $\text{recorded_from} \leq a < \text{recorded_to}$. Nếu ứng dụng lưu $\text{created_at}/\text{expired_at}$ hoặc tên khác, phải chứng minh mapping ngữ nghĩa bằng test dữ liệu sửa.

Một path phục vụ kết luận đồng thời phải có khoảng giao của các fact tương thích với thời điểm query. Ví dụ, relation “thuộc vùng A” tháng 7 và “tham gia dịch vụ B” tháng 9 không chứng minh người đó đồng thời thỏa cả hai tháng 8. Query mô tả chuỗi sự kiện theo thời gian có semantics khác và không nên bị ép vào cùng phép giao.

Các test bắt buộc gồm hiệu lực tương lai, event đến muộn, bản sửa quá khứ, hai nguồn bất đồng, khoảng chồng lấn, fact bị rút lại, timezone, thiếu ngày và biên ngày hiệu lực. Cần policy rõ về ưu tiên nguồn, giữ unresolved conflict và abstain. Không tự chọn “đến sau là đúng hơn” cho mọi domain chỉ vì một framework có cơ chế invalidation.

8.3. Hồ sơ hành vi phải có thể tái lập

Lớp memory đề xuất	Ví dụ tổng hợp	Yêu cầu provenance và cập nhật
Observed event	Hoàn tất chuyển, đổi vùng, tiếp nhận phản ánh	Event ID, source, event time, ingestion time, correction link
Validated fact	Trạng thái dịch vụ trong một khoảng	Subject/relation/object, valid/recorded intervals, evidence IDs
Derived measure	Tỷ lệ sự kiện loại Y trong 30 ngày	Numerator, denominator, window, quy tắc loại trừ, version code
Profile summary	Mô tả các xu hướng đã quan sát	Source set, thời điểm tính, phạm vi, cách drill-down
Hypothesis	Khả năng cần hỗ trợ thêm	Nhãn suy diễn, uncertainty, expiry, người/quy trình kiểm tra

Đây là schema logic đề xuất, chưa phải schema của GSM. A-Mem hỗ trợ ý tưởng dynamic linking; Hindsight hỗ trợ tách observation/opinion khỏi evidence. Không sử dụng điểm hội thoại của hai hệ để khẳng định có thể đánh giá hành vi tài xế chính xác. [\[2\]](#) [\[12\]](#)

Consolidation nên phục vụ retrieval: summary ngắn dẫn tới tập event liên quan. Aggregate đòi hỏi đủ dữ liệu nên tính bằng truy vấn xác định hoặc batch job, không lấy trung bình trên top-k event gần query. Summary phải được đánh dấu stale khi event nguồn bị sửa. Raw event có thể chuyển cold storage, nhưng cần pointer và chính sách giữ lịch sử đủ cho benchmark/tác vụ.

9. Kết hợp document và KG — RQ5

9.1. Routing trước truy xuất, kiểm tra độ đủ sau truy xuất

Dạng query	Route khởi đầu	Khi nào mở rộng
Hỏi nội dung SOP, không nhắc trạng thái tài xế	Document	Có điều kiện cần fact nghiệp vụ để trả lời

Dạng query	Route khởi đầu	Khi nào mở rộng
Hỏi fact/trạng thái có ID và thời điểm	KG + source event	Cần giải thích quy tắc hay ý nghĩa nghiệp vụ
Hỏi quyết định áp dụng chính sách cho người cụ thể	Cả hai	Thiếu bridge hoặc điều khoản ngoại lệ
Hỏi tổng hợp số lượng/tỷ lệ	Structured query/aggregate + docs định nghĩa	Cần giải thích tiêu chí hoặc kiểm tra nguồn
Mơ hồ về ID, ngày hoặc scope	Resolve/clarify hoặc retrieval thăm dò giới hạn	Chỉ tiếp khi entity/time đã đủ xác định

Đây là routing đề xuất. Ban đầu nên dùng rule/metadata để kiểm tra, rồi mới so với LLM router. Với query policy-driver, chạy hai nguồn song song khi chúng độc lập; nếu policy query phải dùng thuộc tính từ KG, thực hiện hai giai đoạn và ghi nhận phần độ trễ phụ thuộc.

9.2. Fusion phải giữ bổ sung bằng chứng

RRF phù hợp để hợp nhất danh sách từ nhiều retriever trên cùng loại đối tượng. Một edge hạng 1 và một chunk hạng 1 không phải hai đáp án có thể thay thế nhau. Đề xuất chuẩn hóa **evidence unit** với source_type, source_id, entity_ids, time_scope, claims_supported, text, provenance và token_cost. Rank trong từng nguồn trước; sau đó chọn bộ unit đáp ứng vai trò bằng chứng.

Với mỗi query, xác định tập yêu cầu như {driver state, eligibility measure, policy rule, exception}. Một bundle có thể gồm ba fact và hai chunks. Heuristic packing ưu tiên phần tăng coverage trên mỗi token, phạt trùng lặp, giữ bridge cần thiết; áp ràng buộc cứng về ID, quyền truy cập và temporal compatibility. Đây là cách thích nghi từ tư tưởng organization/subgraph selection, không phải thuật toán đã chứng minh tối ưu cho GSM.

Graph hướng dẫn text: dùng policy ID, region/service và thời gian để thu hẹp document candidates.

Text kiểm tra graph: lấy đúng điều khoản và episode nguồn để kiểm tra fact được trích/diễn giải.

Một assertion do LLM tạo không được xem là nguồn độc lập bổ sung cho chính đoạn text đã tạo ra nó.

9.3. Context cuối cần đủ để agent hành động có căn cứ

Đề xuất context gồm thời điểm và entity đã xác định, fact phù hợp có source ID, quy tắc/ngoại lệ gốc, mâu thuẫn hoặc thiếu dữ liệu, và bảng ánh xạ claim-evidence ngắn. Dùng cùng ngân sách 4.000 token làm mốc thử ban đầu; thử 2.000/8.000 trên dev subset để đo độ nhạy, không nhân toàn bộ ma trận thí nghiệm.

Nếu evidence thiếu, một lượt retrieval bổ sung có thể cứu query; nếu dữ liệu không tồn tại, agent phải nói không đủ thông tin. Đánh giá task success bằng công cụ mô phỏng và trạng thái đích xác định.

Câu trả lời đúng với citation sai vẫn là lỗi groundedness.

10. Scalability — RQ6

10.1. Những gì literature thực sự kiểm tra

Nguồn	Quy mô / phép đo đáng chú ý	Kết luận được phép
Zep	LongMemEval S: 500 câu, history khoảng 115k tokens; response time trong thiết lập tác giả	Memory QA ở context dài; chưa chứng minh multi-tenant driver QPS

Nguồn	Quy mô / phép đo đáng chú ý	Kết luận được phép
RAPTOR	Scale appendix tới khoảng 78k input tokens	Có khảo sát tăng kích thước tài liệu; chưa phải triệu chunks
GraphRAG	Khoảng 1m và 1,7m tokens; 1.669/3.197 chunks trong hai corpus	Chi phí construction và global summarization có thể đáng kể
LightRAG	Corpus khoảng 0,6–5m tokens	So sánh RAG trên corpus text; thiếu bằng chứng SLA enterprise
HippoRAG 2	LV 22.849 passages, graph vài triệu edges	Graph expansion có thể lớn; benchmark phần cứng phải đọc kèm
G-Retriever	Graph QA từ graph ứng viên theo câu hỏi	Giảm context không đồng nghĩa search toàn KG lớn
TG-RAG	480 tài liệu, 1,58m tokens; protocol thêm 96 tài liệu	Có bằng chứng update theo đợt ở quy mô corpus đã nêu
GFM-RAG	60 KG, 14m triples, 700k docs dùng huấn luyện	Training scale; không phải số truy vấn phục vụ hoặc online latency
PLAID	Thử nghiệm tới 140m passages	Bằng chứng mạnh hơn cho scale text IR; không bao gồm driver KG

Các số là thông số tác giả công bố, không phải chạy lại trong dự án. Nguồn tương ứng: [\[1\]](#) [\[7\]](#) [\[8\]](#) [\[9\]](#) [\[5\]](#) [\[10\]](#) [\[11\]](#) [\[18\]](#) [\[16\]](#)

10.2. Mô hình chi phí để đo, không coi là benchmark

Ký hiệu N là số chunks, V nodes, E edges, d chiều embedding, k số ứng viên, b branching factor, h số hop và I số iteration propagation.

Thành phần	Phân tích định tính	Cách giới hạn trong prototype
Dense index	Raw vector storage $O(Nd)$; ANN thêm overhead và trade-off recall	Một vector/chunk trước; đo RAM/disk/build time và ANN recall
Lexical/sparse	Theo tổng posting lists; query tùy tần suất term và pruning	Metadata/time filter, kiểm tra tokenizer tiếng Việt
Reranker	Cost tăng theo số candidate và chiều dài cặp query-text	Cap k ; batch; đo CPU/GPU/API riêng
Graph traversal	Fan-out không kiểm soát có thể tăng gần b^h trước khi dedup	Typed relation, degree cap, scope/time pruning
PPR	Full-graph iteration phụ thuộc E và I ; local approximation thay đổi behavior	Giới hạn candidate graph; benchmark accuracy loss
LLM extraction	Tăng theo lượng text và số lần xử lý lại	Structured ingestion, batching có kiểm soát, retry/idempotency
Consolidation	Update có thể lan sang link/summary/ancestor	Dependency tracking, cửa sổ, background jobs

Thành phần	Phân tích định tính	Cách giới hạn trong prototype
Multi-hop agent	Độ trễ cộng theo các bước phụ thuộc	Một follow-up tùy điều kiện, deadline và stop rule

Ví dụ tính dung lượng, **không phải kết quả đo**: 1 triệu vectors $\times$ 1.024 chiều $\times$ 4 bytes = 4,096 GB theo hệ thập phân; 10 triệu tương ứng 40,96 GB. Chưa gồm ANN, metadata, graph, text, replicas, runtime và model weights. Với late interaction, cần nhiều vector cho mỗi passage nên không thể dùng phép tính một vector/chunk này.

Không giả định ANN có query time $O(\log N)$ cho mọi dữ liệu hoặc graph lookup luôn $O(1)$. Phải đo phân phối degree, skew và hard negatives. Những tài xế nhiều sự kiện và node policy có degree lớn có thể chi phối tail latency.

10.3. Partition, update và operational scope

Graphiti group_id là cơ chế namespacing logic; docs cảnh báo fragmentation và mô tả query theo nhóm. Không suy ra physical sharding, authorization enforcement hoặc performance isolation chỉ từ field này. ^[30]

Đề xuất chọn tenant hoặc miễn truy cập làm scope theo schema; không mặc định một graph/namespace cho mỗi driver nếu cần quan hệ liên tài xế hoặc policy dùng chung. Đưa ID, relation type, thời gian và source version vào index/query plan. Kiểm tra quyền ở lớp dữ liệu và retrieval, trước khi evidence vào context.

Đo riêng: extraction throughput, structured insert throughput, embedding build, graph commit, time-to-searchable, query latency, rerank và generation. Đối với telemetry/event lớn, chạy stress test graph bằng dữ liệu có cấu trúc trước; chạy extraction LLM trên mẫu đại diện rồi báo chi phí riêng. Không nhân tuyến tính mẫu nhỏ để tuyên bố ingestion production đã được chứng minh.

Cache phải gắn với entity/time/scope, document version và graph snapshot hoặc invalidation key. Query lịch sử known-as-of và query “hiện tại” cần chính sách cache khác nhau. Theo dõi tombstone/invalidated edge growth, không chỉ dữ liệu mới.

11. Kế hoạch đánh giá có thể thực hiện

11.1. Hai trục: chất lượng có gold và stress có tải

Benchmark công khai cho khả năng so sánh, synthetic GSM-like cho logic schema. Không nguồn nào tự thay thế được production validation. MuSiQue/Hotpot thích hợp multi-hop; LongMemEval có temporal/update/abstention; ECT-QA có protocol tăng corpus; BRIGHT cho reasoning retrieval; CRAG cung cấp câu hỏi và môi trường có nguồn web/KG. [\[4\]](#) [\[6\]](#) [\[22\]](#) [\[23\]](#) [\[11\]](#) [\[25\]](#)

Phạm vi đề xuất để vừa 288 giờ: một public multi-hop subset, LongMemEval S và một temporal update subset; CRAG/BRIGHT chỉ dùng diagnostic nhỏ khi còn ngân sách. Chọn một bộ multi-hop, không tái lập cả bốn bộ của IRCoT. Khi dùng subset, lưu ID, seed, corpus construction và version; không đặt tên kết quả như official full-benchmark score.

Bộ đánh giá đề xuất	Phạm vi khởi đầu	Vai trò
GSM-like dev	200 query từ schema tổng hợp	Tune k, routing, thời gian, context budget
GSM-like test đóng băng	400 query: 80 policy, 80 state, 80 hybrid, 60 multi-hop, 50 update/correction, 50 thiếu/mâu thuẫn	So sánh chính; nhóm chính không trùng, có nhãn phụ thời gian/độ khó
Public diagnostic	Khoảng 100 query mỗi họ được chọn	Kiểm tra cơ chế không chỉ học template synthetic
LongMemEval S đầy đủ	500 query, chỉ baseline và cấu hình cuối nếu ngân sách cho phép	Đối chiếu memory/update với protocol công khai
Human review	Tối thiểu 40 scenario trước đóng băng, thêm mẫu lỗi bất đồng judge	Kiểm tra nghĩa nghiệp vụ giả lập và gold evidence

Các con số trên là đề xuất ngân sách, chưa phải mẫu đã tạo. Có thể giảm số cấu hình hoặc số query công khai sau pilot, nhưng không giảm tập test sau khi xem kết quả để làm đẹp điểm.

11.2. Sinh synthetic có ground truth độc lập

Tạo một event ledger và policy engine đơn giản làm nguồn sự thật: driver IDs, thuộc tính theo khoảng thời gian, sự kiện, version chính sách, điều kiện và ngoại lệ. Gold answer và gold evidence tính từ ledger/quy tắc. Sau đó mới sinh text SOP và diễn đạt query đa dạng từ dữ liệu đó. Nếu dùng LLM viết câu, LLM không phải nguồn duy nhất quyết định đáp án.

Chia train/dev/test theo driver, policy family và thời gian; giữ một tập kết hợp mới để thử generalization. Không để cùng scenario chỉ đổi tên xuất hiện ở cả dev và test. Tạo distractors có cùng thuật ngữ nhưng sai vùng, ngày, service hoặc driver. Tách các nguồn document và graph để tránh câu trả lời luôn xuất hiện nguyên văn trong một nguồn.

Gold evidence là tập các phương án đủ bằng chứng: có thể nhiều đoạn tương đương cùng chứng minh một điều khoản. Ghi cả minimal evidence set và nguồn thay thế chấp nhận được. Kiểm tra bằng counterfactual: đổi một fact điều kiện thì đáp án có đổi đúng không; đổi fact không liên quan thì đáp án phải giữ. Test no-answer phải bao gồm missing data thật, mâu thuẫn chưa giải quyết và câu ngoài phạm vi.

Hai biến thể ingestion: gold KG được dựng trực tiếp từ ledger; extracted KG được dựng từ cùng events đã biểu diễn thành text. Chênh lệch giữa hai biến thể giúp phân biệt extraction/resolution error với retrieval error.

11.3. Metrics phải nối retrieval với agent outcome

Tầng	Metrics cần đo	Diễn giải
Construction	Entity merge/split error, fact precision/recall, temporal interval correctness	Graph sai không được quy thành reader failure
Retrieval	Recall@k, nDCG@k, precision; complete evidence-set recall tại token budget B	Có lấy đủ một phương án chứng minh đáp án hay không
Temporal	Wrong-time evidence rate, known-as-of accuracy, conflict handling	Lấy nhiều fact đúng chủ đề nhưng sai thời gian vẫn thất bại
Context	Số token, evidence coverage, redundancy, citation coverage	So sánh ở cùng budget và source availability
Reader QA	EM/F1 nơi phù hợp; rubric correctness; citation precision/recall; abstention	Không dùng duy nhất LLM judge preference
Agent	Task success trên state/tool simulator; hành động dựa trên điều kiện đúng	Đo tác vụ thật trong mô phỏng, không chỉ đoạn trả lời
Serving	p50/p95/p99; throughput đạt được; timeout/error rate	Đo theo concurrency, cold/warm cache và hardware
Update	Ingestion records/s; freshness lag; old/new query scores; correction latency	Học mới, không phá cũ, sửa được quá khứ
Cost	Tokens/calls/API cost hoặc GPU time; RAM/disk/peak build	Tách offline, online và giá/thiết lập tại ngày chạy

Định nghĩa complete evidence-set recall: một query đạt nếu context chứa ít nhất một tập bằng chứng đủ theo gold. Khác với fact recall trung bình, metric này phạt việc tìm được 3 trong 4 thành phần nhưng thiếu điều khoản quyết định. Khi có nhiều gold set, không bắt hệ thống phải tìm mọi cách chứng minh.

Oracle quan trọng: chạy reader với gold context ở cùng budget để đo trần; chạy retrieved context với reader cố định; chạy gold KG và extracted KG. Nếu gold context vẫn sai, ưu tiên sửa reader/tool semantics; nếu retrieval recall tăng nhưng QA không tăng, phân tích noise, bridge loss, mâu thuẫn và vị trí evidence trước khi thêm retriever.

11.4. Thí nghiệm tối thiểu, có ablation

Cấu hình	Thay đổi so với hàng trước	Câu hỏi cần trả lời
C0	Baseline Graphiti + document retrieval hiện có, ghi rõ version	Điểm xuất phát thực tế là gì?
C1	Document BM25+dense+RRF+bounded rerank	Baseline IR mạnh đã giải quyết bao nhiêu lỗi?
C2	ID/time contract, typed filtering và kiểm tra khoảng	Temporal correctness có tăng với chi phí chấp nhận được?
C3	Bounded query-conditioned graph expansion + source mapping	Có lấy thêm bridge/required evidence?

Cấu hình	Thay đổi so với hàng trước	Câu hỏi cần trả lời
C4	Evidence bundle selection ở cùng token budget	Context tốt hơn có tăng QA/task success?
C5	Một follow-up retrieval có điều kiện	Lợi ích có xứng số calls và tail latency?

Đây là chuỗi cấu hình, không đủ để suy ra mọi interaction. Trên dev subset, bỏ riêng reranker và bỏ riêng packing ở cấu hình cuối để chẩn đoán hai thành phần chính; không mở full factorial. C5 là tùy chọn nếu C4 đã ổn định.

Giữ cố định reader, embedding khi so thành phần không liên quan, prompt, corpus snapshot và budget. Dùng paired comparisons trên cùng query; báo confidence interval qua bootstrap theo scenario/driver để tránh coi paraphrase tương quan là mẫu độc lập. Tách kết quả theo category, không chỉ điểm trung bình. Nếu dùng LLM judge, lưu model/prompt/version, kiểm tra blind pair order và lấy human audit ở mẫu bất đồng.

11.5. Stress tiers và tiêu chí đi tiếp

Tier tải đề xuất	Chunks	Drivers	Events	Mục đích
S	10k	1k	100k	Debug và đo độ đúng
M	100k	10k	1m	Profiling, fan-out và update
L, stretch	1m	100k	10m	Stress cấu trúc nếu hardware/ngân sách cho phép

Đếm V/E thực tế sau ingestion; không coi events bằng edges. Với mỗi tier, thử concurrency 1/8/32 khi máy chịu được; công bố throughput đo được thay vì suy QPS bằng nghịch đảo p95. Giữ query set chất lượng cố định và thêm distractors để đo suy giảm retrieval khi memory lớn. Tách thử một graph lớn với nhiều graph nhỏ vì hai phân bố tải khác nhau.

SLA production, số người dùng và hardware chưa được cung cấp. Tuần 1 phải chốt retrieval latency budget, freshness budget và chi phí trần với mentor. Tiêu chí đề xuất là cấu hình cuối tăng complete evidence recall và QA có ý nghĩa trên paired test, không tăng lỗi thời gian/nhầm người, đồng thời nằm trong ngân sách đã chốt. Mức tăng 5 điểm phần trăm có thể dùng làm mục tiêu kỹ thuật ban đầu, không phải mức tăng được literature bảo đảm; báo CI và mức thực tế ngay cả khi không đạt.

12. Khoảng trống và kết quả nghiên cứu không đồng nhất

Retrieval tốt hơn không bảo đảm QA tốt hơn. IRCoT có cấu hình tăng recall nhưng không tăng QA; KG²RAG cho thấy expansion rộng hơn có thể tăng recall cùng context dài hơn mà không cải thiện đáp án tương ứng. Mem0 còn báo full-context đạt điểm judge cao hơn các biến thể memory trong thiết lập của họ, dù memory giảm latency/token cost. Vì vậy tối ưu chất lượng và hiệu quả là một frontier, không có cấu hình thắng mọi chiều. ^{[6] [4] [19]}

Các bảng memory không đồng nhất về backbone và judge. Zep, A-Mem, Mem0 và Hindsight có protocol, model, category và metric khác nhau; một số dùng số liệu của nghiên cứu trước. Không tạo league table accuracy từ các headline. Dùng chúng để chọn cơ chế/ablation, rồi chạy baseline chung. ^{[1] [2] [19] [12]}

Temporal correctness còn thiếu kiểm thử sát vận hành. Nhiều paper xử lý timestamp hoặc fact mới, nhưng chưa chứng minh đồng thời late correction, source priority, valid-at và known-as-of trên

policy-driver join. TG-RAG mở hướng protocol update hữu ích; IA-RAG nghiên cứu interval reasoning nhưng còn mới và chỉ được kiểm tra ở mức abstract/repository trong review này. ^{[11] [27]}

Graph-document alignment là khoảng trống sát GSM. Phần lớn graph-enhanced RAG xây graph từ chính corpus; nhiều agent-memory benchmark bắt đầu bằng hội thoại. Một operational KG độc lập, policy có version, join bằng thuộc tính và quyền truy cập là tổ hợp ít được đánh giá trực tiếp trong các nguồn đã xem. Đây là cơ hội R&D tốt hơn việc thêm một framework nữa.

Scale bằng số token không giải quyết update và tail latency. Thiếu thông tin hardware, concurrency, index layout, độ skew hoặc graph size thì không thể kiểm chứng “production scalable”. Công việc sáu tuần nên để lại công cụ đo và đường cong giới hạn, ngay cả khi tier lớn nhất không chạy được.

External validity chưa được giải quyết. Synthetic data có thể quá sạch, relation distribution quá đều, policy logic quá ngắn hoặc cách hỏi giống template. Public English benchmark không chứng minh multilingual robustness. Báo cáo cuối dự án phải nêu phạm vi đã kiểm chứng và điều kiện cần một pilot có dữ liệu đại diện sau này.

13. Ưu tiên công nghệ và rủi ro

Ước lượng effort dưới đây là phần kỹ thuật trực tiếp và có thể chống lẩn với evaluation; không cộng riêng với kế hoạch 288 giờ ở mục 14. Expected benefit là giả thuyết nghiên cứu, không phải uplift dự báo.

Hướng	Lợi ích kỳ vọng / effort	Rủi ro scale và thực nghiệm	Quyết định
Pin/audit Graphiti và temporal query contract	Đúng người/thời gian; 16–24h	Schema/semantics mơ hồ; test history khó	Tier 1
Metadata + BM25/de nse/RRF/rerank	Recall và precision tài liệu; 24–36h	Rerank latency; tiếng Việt; baseline có thể đã mạnh	Tier 1
Gold ledger + harness + oracle	Chẩn đoán đúng nguyên nhân; 32–48h	Synthetic bias, gold không độc lập	Tier 1
Fact-source mapping + bounded expansion	Bổ sung bridge evidence; 20–28h	High-degree nodes, sai join, graph noise	Tier 1
Evidence packing + citations	Tăng độ đủ ở cùng budget; 16–24h	Reranker bỏ ngoại lệ; gold sets không đầy đủ	Tier 1
Scale/update instrumentation	Định lượng giới hạn; 24–32h	Hạ tầng thiếu; LLM quota; thay đổi cache	Tier 1
Một follow-up kiểu IRCot	Cứu query thiếu bước; 8–16h	Tuần tự, query drift, thêm token	Tier 2
Contextual prefix / parent-child	Tìm đoạn khó hiểu độc lập; 8–16h	Prefix sai hoặc stale	Tier 2; chọn trước full RAPTOR
Local PPR hoặc PCST heuristic	Graph selection có cấu trúc; 16–24h	Graph nỏ; connectedness không đúng mọi query	Tier 2; tối đa một lựa chọn

Hướng	Lợi ích kỳ vọng / effort	Rủi ro scale và thực nghiệm	Quyết định
Một trong SPLADE/CoBERT/ReasonIR	Giảm lexical/reasoning mismatch; 16-24h	Index/model lớn, domain mismatch	Tier 2 nếu profiling chỉ ra nhu cầu
A-Mem-style linking/consolidation	Recall event dài hạn; 12-24h	Write amplification và summary drift	Tier 2 sau temporal correctness
RAPTOR/full global GraphRAG	Tổng hợp corpus/tài liệu dài	Rebuild và LLM summary cost	Tier 3; chỉ triển khai khi có query tương ứng
MemGPT/Letta, Hindsight, Mem0 toàn stack	Hiểu memory orchestration	Tăng phạm vi tích hợp, trùng baseline	Tier 3; đọc và tái dùng cơ chế
GFM-RAG / G-Retriever full training	Learned graph retrieval	Cần dữ liệu, GPU, tuning và adapter	Tier 3 cho đọc; training ngoài phạm vi

Ngoài phạm vi: training foundation model mới, tối ưu hạ tầng phân tán lớn trước khi biết bottleneck, tái tạo mọi benchmark, thay đổi LLM backbone liên tục để tăng điểm, và suy ra kết luận hành vi thật của tài xế từ dữ liệu giả lập. Các hướng này không giúp hoàn tất một hệ thống coherent trong sáu tuần.

Stop rule: đến cuối tuần 3, chỉ giữ tối đa hai cải tiến tùy chọn ngoài Tier 1. Dừng một hướng nếu không cải thiện loại lỗi mục tiêu sau hai vòng cấu hình có kiểm soát, nếu vi phạm budget đã chốt, hoặc nếu cần thay đổi phần lớn memory stack. Lưu kết quả âm để làm rõ lựa chọn cuối.

14. Kế hoạch sáu tuần — 288 giờ

Mỗi tuần 6 ngày × 8 giờ = 48 giờ. Mục tiêu là một agent end-to-end với retrieval có thể đo và giải thích, không phải 27 bản tái lập paper.

Tuần	Phân bổ 48 giờ	Deliverable và gate
1	16h đọc core/schema; 16h dựng baseline và log; 16h query contract, ledger và pilot gold	C0 chạy end-to-end; schema mapping, ID/time semantics, hardware/cost/latency budget được ghi rõ
2	12h normalization/chunk/metadata; 12h hybrid retrieval; 12h rerank; 12h dev evaluation	C1 + lexical/dense ablation; biết lỗi retrieval tài liệu chủ đạo
3	10h entity resolution; 16h temporal/update; 12h bounded graph retrieval; 10h evaluation	C2/C3; vượt test nhằm người, thời gian, late correction; chốt tối đa hai tùy chọn
4	12h graph-policy joins; 12h evidence packing; 8h optional follow-up; 16h ablation và agent integration	C4, có thể C5; query hybrid có evidence đầy đủ và citation; đóng băng cấu hình trước final test
5	20h scale grid; 12h update/correction/failure; 12h profiling/tuning; 4h quyết định giới hạn	Đường cong quality/latency/storage/freshness ở S/M; L nếu đủ điều kiện; không còn bottleneck chưa định danh
6	12h final frozen evaluation; 8h thống kê/error analysis; 12h demo/reproducibility; 12h báo cáo; 4h handover	Một hệ thống demo, cấu hình có thể chạy lại, bảng kết quả/CI, giới hạn và khuyến nghị pilot

Trong tuần 1, cần lấy từ mentor/schema: khóa định danh; loại event và relation; policy versioning; timezone; source-of-truth; phân quyền; các tác vụ agent cần làm; giả định tải và phần cứng. Đây là đầu vào triển khai cần xác nhận khi bắt đầu dự án, không phải điều kiện để hoàn thành literature review này.

Artifact kỹ thuật của dự án nên gồm configuration manifest, dataset generator + seeds, query/evidence gold, adapters document/KG, eval harness, raw results, error taxonomy và demo scenario. Cố định dependency versions và lưu request/response metadata đủ để truy vết chi phí mà không phụ thuộc vào dashboard một vendor.

Demo cuối: một query SOP; một historical driver state; một policy-driver decision có ngoại lệ; một late correction làm đổi đáp án đúng; một no-answer; một load report. Demo phải dùng cùng pipeline được benchmark, tránh đường xử lý riêng chỉ dành cho trình diễn.

15. Thứ tự đọc đề xuất

Foundation — 4 mục

1. **BEIR [21]**. Đọc vì nó giúp thiết kế baseline và hiểu vì sao cần đánh giá retriever ngoài một dataset thuận lợi.
2. **RRF [26]**. Đọc vì đây là phép fusion đủ đơn giản để kiểm tra và triển khai trước các mô hình phức tạp.
3. **MemGPT [3]**. Đọc vì cần phân biệt context/memory management với chất lượng retrieval.
4. **LongMemEval [23]**. Đọc vì temporal/update/abstention phải được biến thành các bài test rõ ràng.

Core — 8 mục

1. **Zep + Graphiti docs [1, 28-32]**. Đọc vì phải hiểu chính xác baseline, khác biệt phiên bản và chi phí hidden trong ingestion/search.
2. **KG²RAG [4]**. Đọc vì nó gần nhất với expansion và organization của bằng chứng graph-text.
3. **BGE-M3 [13]**. Đọc vì cần chọn và kiểm tra một multilingual baseline có thể dùng sẵn.
4. **IRCoT [6]**. Đọc vì một số bằng chứng chỉ tìm được sau bước trung gian và QA không tự tăng theo recall.
5. **HippoRAG 2 [5]**. Đọc vì associative graph retrieval có lợi ích rõ nhưng graph growth cần định lượng.
6. **TG-RAG [11]**. Đọc vì protocol trước/sau update có thể chuyển thành test temporal driver memory.
7. **A-Mem [2]**. Đọc vì linking/evolution có thể được tái dùng độc lập cho event memory.
8. **BRIGHT [22]**. Đọc vì cần phân biệt truy xuất gần ngữ nghĩa với truy xuất bằng chứng cho suy luận.

Advanced / optional — 8 mục

1. **G-Retriever [10]**. Đọc vì lựa chọn subgraph phải bảo toàn bridge và context budget.
2. **Hindsight [12, 34]**. Đọc vì memory cần tách fact, observation và inference.
3. **LightRAG [9]**. Đọc vì entity-level và topic/relation-level search là hai đường bổ sung hữu ích.
4. **RAPTOR [7]**. Đọc khi tài liệu dài và câu hỏi tổng hợp vẫn khó sau parent-child baseline.
5. **GraphRAG + local search [8, 33]**. Đọc khi cần đối chiếu local evidence retrieval với global summarization.
6. **ColBERTv2 [15]**. Đọc khi reranking một vector/chunk chưa đủ phân biệt đoạn gần nhau.
7. **PLAID [16]**. Đọc cùng ColBERT để hiểu chi phí index và khả năng scale thực nghiệm.
8. **ReasonIR [17]**. Đọc khi reasoning mismatch đã được chứng minh là bottleneck và có tài nguyên chạy checkpoint.

SPLADE, Contextual Retrieval, Mem0 và CRAG là tài liệu tra cứu theo thí nghiệm; GFM-RAG và IA-RAG dành cho mở rộng sau dự án. Không cần đọc tuyến tính toàn bộ bibliography trước khi dựng baseline.

16. Khuyến nghị cuối cùng

Document side: triển khai baseline lexical+dense multilingual, rerank có giới hạn, metadata/version/time filters và source text nguyên vẹn. Thử parent context trước hierarchical LLM indexing; đo trực tiếp tiếng Việt và mã nghiệp vụ.

KG side: giữ Graphiti; ưu tiên business IDs, provenance, temporal query contract và bounded typed traversal. Lưu event gốc, fact có hiệu lực và derived profile dưới dạng có thể tái lập. Không để summary hoặc fuzzy resolution ghi đè lịch sử đã xác nhận.

Hybrid layer: route theo loại tác vụ; dùng explicit applicability joins giữa driver KG và policy corpus; chọn evidence bundle đáp ứng điều kiện đủ dưới token budget. Cho phép một follow-up khi thiếu bridge, có stop rule và latency cap.

Scale strategy: đo riêng text index, graph growth, extraction và serving; giới hạn fan-out/candidate count; cập nhật cục bộ; cache theo scope/time/version; stress với dữ liệu phân bố lệch và sửa lịch sử. Chỉ công bố scale đã chạy thành công cùng hardware/concurrency.

Evaluation: tối ưu đồng thời complete evidence recall, temporal correctness, QA/agent task success và chi phí. Dùng public benchmarks để kiểm tra cơ chế, synthetic có latent ledger để kiểm tra

schema, oracle và ablation để biết nguyên nhân. Kết quả sáu tuần nên là một recommendation có bằng chứng và giới hạn rõ, tạo cơ sở cho pilot với dữ liệu đại diện khi được cung cấp.

17. Sources

Các số tham chiếu trong nội dung trở tới paper hoặc tài liệu chính thức dưới đây. Publication status được ghi theo nguồn chính thức kiểm tra đến 14/09/2026. Repository hiện hành có thể khác bản thí nghiệm; cần pin version khi triển khai.

1. Rasmussen và cộng sự (2025). **Zep: A Temporal Knowledge Graph Architecture for Agent Memory**. arXiv technical report; author/vendor evaluation. [Paper](#) · [Full text](#) · [Code Graphiti](#).
2. Xu và cộng sự (2025). **A-MEM: Agentic Memory for LLM Agents**. NeurIPS 2025. [Proceedings](#) · [Paper](#) · [Code](#).
3. Packer và cộng sự (2023; revised 2024). **MemGPT: Towards LLMs as Operating Systems**. arXiv. [Paper](#) · [Successor implementation](#).
4. Zhu và cộng sự (2025). **Knowledge Graph-Guided Retrieval Augmented Generation**. NAACL 2025. [Paper/proceedings](#) · [Code KG²RAG](#).
5. Jiménez Gutiérrez và cộng sự (2025). **From RAG to Memory: Non-Parametric Continual Learning for Large Language Models**. ICML 2025, HippoRAG 2. [Paper](#) · [Code](#).
6. Trivedi và cộng sự (2023). **Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions**. ACL 2023. [Paper/proceedings](#) · [Code IRCoT](#).
7. Sarthi và cộng sự (2024). **RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval**. ICLR 2024. [Paper](#) · [Proceedings PDF](#) · [Code](#).
8. Edge và cộng sự (2024; revised 2025). **From Local to Global: A Graph RAG Approach to Query-Focused Summarization**. arXiv technical report; venue chưa xác minh. [Paper](#) · [Code GraphRAG](#).
9. Guo và cộng sự (2024 preprint; 2025 proceedings). **LightRAG: Simple and Fast Retrieval-Augmented Generation**. Findings of EMNLP 2025. [Paper/proceedings](#) · [Code](#).
10. He và cộng sự (2024). **G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering**. NeurIPS 2024. [Proceedings](#) · [Paper](#) · [Code](#).
11. Han và cộng sự (2025). **RAG Meets Temporal Graphs: Time-Sensitive Modeling and Retrieval for Evolving Knowledge**. arXiv, TG-RAG. [Paper](#) · [Code/data](#).
12. Latimer và cộng sự (2025). **Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects**. arXiv technical report. [Paper](#) · [Code](#). Companion demo: [34].
13. Chen và cộng sự (2024). **BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation**. Findings of ACL 2024. [Paper/proceedings](#) · [Code FlagEmbedding](#).
14. Formal, Piwowarski và Clinchant (2021). **SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking**. SIGIR 2021. [Paper](#) · [Code](#).
15. Santhanam và cộng sự (2022). **ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction**. NAACL 2022. [Paper/proceedings](#) · [Code](#).
16. Santhanam và cộng sự (2022). **PLAID: An Efficient Engine for Late Interaction Retrieval**. CIKM 2022. [Paper](#) · [DOI](#) · [Code](#).
17. Shao và cộng sự (2025). **ReasonIR: Training Retrievers for Reasoning Tasks**. COLM 2025; review này đối chiếu metadata, abstract và code, chưa kiểm toán full appendix. [Paper](#) · [OpenReview](#) · [Code](#).
18. Luo và cộng sự (2025). **GFM-RAG: Graph Foundation Model for Retrieval Augmented Generation**. NeurIPS 2025; arXiv v3 ghi acceptance. [Paper](#) · [Code](#).
19. Chhikara và cộng sự (2025). **Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory**. arXiv; author/vendor evaluation. [Paper](#) · [Code](#).
20. Anthropic (2024). **Introducing Contextual Retrieval**. Engineering report, không peer reviewed. [Report và implementation links](#).

21. Thakur và cộng sự (2021). **BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models**. NeurIPS Datasets and Benchmarks 2021. [Proceedings](#) · [Code/data](#).
22. Su và cộng sự (2024 preprint; 2025 venue). **BRIGHT: A Realistic and Challenging Benchmark for Reasoning-Intensive Retrieval**. ICLR 2025. [Paper](#) · [Code/data](#).
23. Wu và cộng sự (2024 preprint; 2025 venue). **LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory**. ICLR 2025. [Paper](#) · [Code/data](#).
24. Maharana và cộng sự (2024). **Evaluating Very Long-Term Conversational Memory of LLM Agents**. ACL 2024, LoCoMo. [Paper](#) · [Code/data](#).
25. Yang và cộng sự (2024). **CRAG: Comprehensive RAG Benchmark**. Paper/benchmark của Meta, KDD Cup 2024 challenge; venue của bài chưa được xác minh trong review. [Paper](#) · [Code/data](#).
26. Cormack, Clarke và Büttcher (2009). **Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods**. SIGIR 2009. [Author-hosted paper PDF](#). Không xác minh repository độc lập của bài.
27. Wang và cộng sự (2026). **IA-RAG: Interval-Algebra-Driven Temporal Reasoning for Dynamic Knowledge Retrieval**. arXiv:2606.06044; chỉ đọc abstract/repository trong review. [Paper](#) · [Code](#).
28. Graphiti maintainers. **Graphiti repository và phân biệt Graphiti/Zep**. [Official repository](#).
29. Graphiti documentation. **Searching the graph**. [Official docs](#).
30. Graphiti documentation. **Graph namespacing**. [Official docs](#).
31. Graphiti documentation. **Adding fact triples**. [Official docs](#).
32. Graphiti documentation. **Communities**. [Official docs](#).
33. Microsoft GraphRAG documentation. **Local Search**. [Official docs](#).
34. Latimer và cộng sự (2026). **Hindsight: Structured Agent Memory that Retains, Recalls, and Reflects**. ACL 2026 System Demonstrations. [Proceedings](#).
35. A-Mem authors. **Evaluation implementation**. [Repository](#).
36. Letta (2024). **MemGPT and Letta**. [Official explanation](#).
37. Letta maintainers. **Letta**. [Repository](#).
38. KG²RAG authors. **KG²RAG**. [Repository](#).
39. HippoRAG authors. **HippoRAG**. [Repository](#).
40. IRCot authors. **IRCot**. [Repository](#).
41. RAPTOR authors. **RAPTOR**. [Repository](#).
42. Microsoft. **GraphRAG**. [Repository](#).
43. LightRAG maintainers. **LightRAG**. [Repository](#).
44. G-Retriever authors. **G-Retriever**. [Repository](#).
45. TG-RAG authors. **Temporal-GraphRAG**. [Repository](#).
46. Hindsight maintainers. **Hindsight**. [Repository](#).